Daily Quant Digest: 2026-10-30

Foundations Restart, day 5. Yesterday we counted groups where order does not matter, learned to read a currency price tag, and met the hash set. Today: sharing out identical things (stars and bars), the forward contract (the simplest derivative of all), and prefix sums, the trick of answering "how much between here and there?" with one subtraction.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 5: counting III: stars and bars, balls into boxes, counting the complement	Easy-Medium
Derivatives	FR Lesson 5: forwards: what they are, the payoff, who uses them	Easy
Algorithms & DS	FR Lesson 5: prefix sums; longest stretch of days with total P&L exactly k	Easy-Medium
Music	Eagles, "Hotel California" (1976)	

1. Probability & Statistics: Sharing Out Identical Things

> **Where we are.** Lesson 2: to count "bad" cases, count the complement or use inclusion-exclusion. Lesson 3: $n!$ ways to line up n different things. Lesson 4: $\binom{n}{k}$ ways to choose a bag of k from n, found by counting queues and dividing by the $k!$ orders each bag can be lined up in. At the end of Lesson 4 we said "divide by $k!$" breaks when items repeat. Today we fix that, and it turns out to be the key to a whole new kind of counting.

1.1 Warm-up: lining up letters that repeat

How many different "words" can you make from the letters of **MISSISSIPPI**? There are 11 letters, so if they were all different the answer would be $11!$. But they are not. There are 4 S's, 4 I's, 2 P's and 1 M.

Use the same idea as Lesson 4. Pretend for a moment the letters have hidden labels: S_1, S_2, S_3, S_4. Then there are $11!$ arrangements. Now peel the labels off. Every real word was counted many times: once for each way of shuffling the 4 S's among their own places (

4!), times the I's (4!), times the P's (2!). Every word is hit **the same** number of times, so we can divide:

$$\frac{11!}{4!\,4!\,2!\,1!} = 34{,}650.$$

Read aloud: "11 factorial, divided by the factorial of each letter's count." Keep this in your pocket: it is exactly what powers today's main tool.

1.2 The central picture: coins and fence posts

A desk head has **7 identical blocks of risk** (say USD 1 million of risk limit each) to share among **3 desks**: Rates, FX and Credit. A desk can get zero. How many ways are there?

The blocks are identical, so all that matters is **how many** each desk gets: 3 + 4 + 0, or 2 + 2 + 3, and so on.

Here is the picture. Lay the 7 blocks in a row as stars, and put **2 fence posts** (bars) among them. Everything left of the first bar goes to Rates, between the bars to FX, right of the second bar to Credit:

Row of symbols	Rates	FX	Credit
★★★ | ★★★★ |	3	4	0
★★ | ★★ | ★★★	2	2	3
| ★★★★★★★ |	0	7	0
★★★★★★★ ||	7	0	0

Every way to share is **one** row of 7 stars and 2 bars, and every row is one way to share. So we only need to count the rows. A row has $7 + 2 = 9$ positions. Choose which 2 of them hold bars; the rest are stars:

$$\binom{9}{2} = 36.$$

That is the whole trick. Sharing becomes "line up a word with repeated letters", which is Section 1.1 (or simply "choose the bar positions", which is Lesson 4).

In the picture	In the maths
The identical blocks (stars)	The n identical items
The desks	The k boxes (or variables)
The fence posts (bars)	$k - 1$ dividers
A row of stars and bars	One solution of $x_1 + \cdots + x_k = n$ with $x_i \geq 0$
Choosing where the bars go	$\binom{n+k-1}{k-1}$

1.3 The formula, one step at a time

In words: to put n identical items into k boxes, you need $k-1$ bars. The row has $n+k-1$ symbols. Choose which $k-1$ of them are bars.

$$\#\{x_1 + x_2 + \cdots + x_k = n,\ \text{each } x_i \geq 0\} = \binom{n+k-1}{k-1}.$$

Symbols. n = number of identical items (stars). k = number of boxes. x_i = how many go into box i. The $\#$ sign means "how many".

Read aloud: "the number of ways to split n into k non-negative parts is n plus k minus 1, choose k minus 1."

With numbers. Our desks: $n = 7$, $k = 3$, so $\binom{9}{2} = \frac{9\times 8}{2} = 36$.

A check by hand. 4 coins into 3 boxes: $\binom{6}{2} = 15$. List them by the first box: first box 4 (1 way: 4,0,0), 3 (2 ways), 2 (3 ways), 1 (4 ways), 0 (5 ways). $1+2+3+4+5 = 15$. Good.

Variant: every box gets at least one. Each desk must get at least one block. Simple fix: **hand out one block to each desk first**. That uses 3 blocks, and the remaining 4 can go anywhere: $\binom{4+2}{2} = \binom{6}{2} = 15$. In general, "at least one each" gives $\binom{n-1}{k-1}$. (In the picture: put the bars only in the 6 gaps **between** stars, never two in the same gap.)

1.4 Upper limits: count the complement

Now the risk manager says: **no desk may get more than 4 blocks**. How many of the 36 shares survive?

Count the **bad** ones and subtract (Lesson 2).

- **Rates gets 5 or more.** Give Rates 5 blocks up front. The other 2 can go anywhere: $\binom{2+2}{2} = 6$ ways.
- Same for FX: 6. Same for Credit: 6.
- **Two desks with 5 or more** each would need 10 blocks. We only have 7, so this never happens and there is no double counting to fix.

Bad shares: $6+6+6 = 18$. Good shares: $36 - 18 = 18$. (I listed all of them by computer: 18.)

If two desks **could** both go over (with more blocks), inclusion-exclusion would add those back, exactly as in Lesson 2.

1.5 Common mistakes

- **Identical or different?** Stars and bars is for **identical** items. 7 different traders split among 3 desks is a different question: each trader picks a desk, $3^7 = 2{,}187$ ways (Lesson 3). Ask: "if I swap two items, is it a different outcome?"

- **The shares are not equally likely.** If 7 blocks are each sent to a random desk, the 36 shares do **not** have equal chances. "All 7 to Rates" happens one way; "3, 2, 2" happens in $\frac{7!}{3!\,2!\,2!} = 210$ ways. Stars and bars counts the **possible splits**; it is not automatically a probability room. Use it as a room only when the splits themselves are equally likely, or when the items really are a bag (as in the interview question below).
- **Off by one.** Count the bars: k boxes need $k - 1$ bars, not k.

1.6 Self-check

1. How many ways can 10 identical chocolates go to 4 children (some may get none)?
2. Same, but every child gets at least one?
3. How many different words can you make from the letters of BANANA?

1.7 Interview question: four dice in a row

(Dice and stars-and-bars questions are common early-round questions at SIG, Optiver and Jane Street. The real test is spotting that an ordered-looking question is a bag in disguise.)

You roll a fair six-sided die 4 times and write down the results in order.

a. What is the probability that the results never go down, that is, each roll is at least the one before it (like 2, 2, 5, 6)?

b. What is the probability that each roll is **strictly** bigger than the one before?

c. What is the probability that the 4 rolls add up to exactly 12?

1.8 Solution

The room. Ordered results: $6^4 = 1{,}296$, all equally likely.

(a) Never going down. Here is the key thought. Take any **bag** of 4 numbers from 1 to 6, repeats allowed, say {2, 5, 2, 6}. There is **exactly one** way to write it in non-decreasing order: 2, 2, 5, 6. So non-decreasing sequences match bags one to one.

Now count bags with stars and bars. A bag is described by how many 1s, how many 2s, …, how many 6s it has. So we share 4 identical "slots" among 6 boxes (the faces):

$$x_1 + x_2 + \cdots + x_6 = 4, \qquad \binom{4+5}{5} = \binom{9}{5} = \binom{9}{4} = 126.$$

$$P(\text{never goes down}) = \frac{126}{1{,}296} = \frac{7}{72} \approx 0.097.$$

(b) Strictly increasing. Now repeats are banned, so a bag is just 4 **different** faces, and each such bag has exactly one increasing order: $\binom{6}{4} = 15$.

$$P = \frac{15}{1{,}296} \approx 0.0116.$$

(c) Sum 12. Write each roll as $1 + y_i$, where y_i is between 0 and 5. Then $y_1 + y_2 + y_3 + y_4 = 12 - 4 = 8$.

- **No upper limit:** $\binom{8+3}{3} = \binom{11}{3} = 165$.
- **Bad: some** $y_i \geq 6$ (a die showing 7 or more, impossible). Give that y_i six up front; 2 are left for all four: $\binom{2+3}{3} = \binom{5}{3} = 10$. Four dice could be the culprit: $4 \times 10 = 40$.
- **Two bad at once** would need $12 > 8$. Impossible.

Good: $165 - 40 = 125$, so $P = \dfrac{125}{1{,}296} \approx 0.096$.

Note: here each **ordered** sequence of 4 rolls is a solution $(y_1, \ldots, y_4)$, and those are equally likely. So stars and bars counts the right room. That is the difference from the mistake in Section 1.5.

Interview trick for (a). Think of the answer to (a) as "pick 4 faces with repeats allowed". The quick formula for a bag of k from n types with repeats is $\binom{n+k-1}{k}$. Say it out loud: "6 plus 4 minus 1 choose 4, that's 9 choose 4, 126."

Sanity check. (b) is a part of (a), and $15 < 126$. Good. For (a) with only 2 rolls, the formula gives $\binom{7}{2} = 21$ out of 36. Direct: 6 doubles plus half of the other 30, that is $6 + 15 = 21$. Good. And I listed all 1,296 outcomes by computer: 126, 15 and 125.

Self-check answers. (1) $\binom{13}{3} = 286$. (2) $\binom{9}{3} = 84$. (3) $\frac{6!}{3!\,2!\,1!} = \frac{720}{12} = 60$.

---

2. Derivatives: The Forward Contract

> **Where we are.** Lesson 2: money later is worth less than money now, and we convert with a discount factor $DF = 1/(1+rt)$. Lesson 3: being long means you gain when the price rises; short means you gain when it falls. Lesson 4: "EURUSD 1.1650" means 1 euro costs 1.1650 dollars, and one pip on EUR 5 million is USD 500. Today we meet the first real derivative.

2.1 The scene

It is 30 October. A Hong Kong company that imports German machines has just signed a deal. It must **pay EUR 5 million in 3 months**, on 30 January. It keeps its books in US dollars (the Hong Kong dollar is pegged to the dollar, Lesson 4).

Today EURUSD is 1.1650, so the machines cost about USD 5.825 million. The finance director's worry: what if the euro jumps to 1.2200 by January? Then the same machines cost USD 6.1 million, and the profit on the deal is gone. The company is not in the business

of betting on currencies. It sells machines.

So the director calls the bank and asks: "**Fix me a rate today for buying EUR 5 million on 30 January.**" The bank quotes **1.1700**. Done. In January, the company will pay USD 5.85 million and receive EUR 5 million, whatever the market does.

That deal is a **forward contract**.

2.2 The central picture: a pre-order with the price written today

Think of pre-ordering a new phone. Today you agree the price and the delivery date. No money changes hands yet. On delivery day you pay the agreed price and get the phone, even if the shop's price has gone up or down in the meantime.

A forward is a pre-order where **both** sides are locked in: you must buy, and the bank must sell.

In the picture	In the forward
The phone	The **underlying**: here, EUR 5 million
The agreed price	The **forward rate** or strike K: 1.1700
Delivery day	**Maturity** (or value date): 30 January
You, the buyer	The **long** side: agrees to buy the base currency
The shop	The **short** side: agrees to sell it
Nothing paid today	A forward costs **zero** to enter

Who is on each side and why?

- **Importers** (paying foreign currency later) buy forwards, like our company.
- **Exporters** (receiving foreign currency later) sell forwards. A Japanese car maker that will receive USD 100 million from US sales in 6 months sells USD forward against yen, to know how many yen it will get.
- **Investors** with foreign bonds sell the foreign currency forward to remove the currency risk from the bond return.
- **Speculators** use forwards to bet on a currency without paying anything up front.
- **The bank** sits in the middle, like the market maker in Lesson 3. It usually does not want the risk, so it hedges, for example by doing the opposite trade with an exporter or in the market.

2.3 The payoff: what you gain or lose at maturity

In words: at maturity, the long side buys at K something that is worth the spot rate S_T in the market. The gain is the difference, times the amount.

$$\text{long payoff} = N \times (S_T - K), \qquad \text{short payoff} = N \times (K - S_T).$$

Symbols. N = notional, the amount of base currency (EUR 5,000,000). K = the forward rate agreed today (1.1700). S_T = the spot rate on the maturity date T (unknown today). The payoff is in the **quote** currency (USD), as with pips in Lesson 4.

Read aloud: "the long gets notional times spot at maturity minus the agreed rate."

With numbers. If EURUSD is 1.2200 in January: $5{,}000{,}000 \times (1.2200 - 1.1700) = 5{,}000{,}000 \times 0.0500 = 250{,}000$. The company gains USD 250,000 on the forward, and the bank loses exactly the same amount. A forward is **zero-sum**: one side's gain is the other side's loss.

If you draw the long payoff against S_T you get a **straight line** through K, going up at 45 degrees. No kinks, no floor. Hold on to that picture: when we meet options, the line will bend.

2.4 Why the hedge works: three possible Januaries

The company's real cost is "buy EUR 5 million at the market rate", plus the forward payoff.

EURUSD on 30 Jan (S_T)	Cost at market	Forward payoff	Total cost
1.1200 (euro weaker)	USD 5,600,000	−USD 250,000	USD 5,850,000
1.1700 (no change from K)	USD 5,850,000	0	USD 5,850,000
1.2200 (euro stronger)	USD 6,100,000	+USD 250,000	USD 5,850,000

The total is **always USD 5.85 million**. The forward's straight line exactly cancels the straight line of the exposure. That is what a hedge is: two lines that tilt in opposite directions add up to a flat line.

The price of certainty. In the first row, the company would have been USD 250,000 better off with no hedge. The forward gives away the good outcomes as well as the bad ones. A hedge locks the price; it does not promise the best price.

Why 1.1700 and not today's 1.1650? The forward rate is not a forecast. It comes from the interest rates of the two currencies, by a no-arbitrage argument. That is Lesson 6. For now, accept that the bank quotes a number for which the contract is worth zero to both sides today.

2.5 How it settles

- **Physical delivery.** On 30 January the company pays USD 5.85 million and receives EUR 5 million. This is what the importer wants: it needs the euros.
- **Cash settlement.** Only the difference changes hands: in the 1.2200 case, the bank pays the company USD 250,000. Currencies that cannot be freely delivered offshore (for example the Korean won, Indian rupee and Taiwan dollar) trade as **non-deliverable forwards (NDFs)**, which always settle in cash, usually in US dollars.

2.6 Before maturity: what is the forward worth?

On day one it is worth zero. A month later, suppose the bank would now quote **1.1800** for the same 30 January date. Our company holds a contract to buy at 1.1700 something the market now prices at 1.1800 for that date. Its gain, **paid in January**, is $5{,}000{,}000 \times 0.0100 = 50{,}000$ USD.

But that money arrives in 60 days, so (Lesson 2) we discount it. With USD rates at 4% (ACT/360):

$$\text{value today} = 50{,}000 \times \frac{1}{1 + 0.04 \times \frac{60}{360}} = \frac{50{,}000}{1.006667} \approx \text{USD } 49{,}669.$$

In general: value of a long forward $= N \times (F_{\text{now}} - K) \times DF$, where F_{now} is today's forward rate for the same date.

2.7 The risks

- **Counterparty risk.** A forward is a private promise between two parties (it trades "over the counter"). If the bank or the client goes bust, the winner may never be paid. Lesson 7 shows how futures exchanges fix this with daily margin.
- **The exposure can change.** If the machine order is cancelled, the company still has to buy EUR 5 million. The hedge has become a bet.
- **Cash and accounting pain.** A hedge that loses on paper can look terrible even when the business underneath is fine.

When it went wrong: Cathay Pacific, 2008. Hong Kong's airline had locked in fuel prices for future years with hedging contracts (cousins of forwards). Oil hit about USD 147 a barrel in July 2008, then collapsed to below USD 40 by December. The airline was now committed to paying far more than the market price, and it reported a loss of around HK$8.5 billion for 2008, driven mostly by fuel hedging losses. The hedges did what forwards do: they gave away the benefit of falling prices. The lesson is to hedge the amount you are sure of, not more.

2.8 Likely interview questions

1. **"You are long EUR 10 million forward at 1.1700. At maturity EURUSD is 1.1550. P&L?"** $10{,}000{,}000 \times (1.1550 - 1.1700) = -150{,}000$, so **−USD 150,000** (150 pips at USD 1,000 a pip).
2. **"Why does a forward cost nothing to enter, when an option costs a premium?"** Both sides have an obligation, and K is set so the contract is fair (worth zero) today. An option gives the buyer a **right** without an obligation, so the buyer has to pay for it.
3. **"A Japanese exporter will receive USD 50 million in 6 months. What does it do?"** It **sells USD forward** against JPY (it is short USDJPY forward). If the dollar weakens, it loses on the dollars it receives and gains on the forward.

---

3. Algorithms & Data Structures: Prefix Sums

> **Where we are.** Lesson 1: count steps, $O(n^2)$ vs $O(n)$. Lesson 2: `vector`, references, loops. Lesson 3: scanning an array once with a running value. Lesson 4: `unordered_map` and `unordered_set`, average $O(1)$ lookups. Today we combine the running value with a hash map.

3.1 The central picture: the car's odometer

You drive from Hong Kong to Guangzhou and stop at several towns. How far is it from town 2 to town 5? You could add up each leg: 2 to 3, 3 to 4, 4 to 5. Or you could look at the **odometer** (the total distance counter) at each town, and do **one subtraction**:

$$\text{distance from town } 2 \text{ to town } 5 = \text{odometer at } 5 - \text{odometer at } 2.$$

A **prefix sum** is the odometer for an array. For daily P&L $a_0, a_1, \ldots$, write the running total at each point:

$$P_0 = 0, \qquad P_{i+1} = P_i + a_i.$$

Read aloud: "P_{i+1} is the total of the first $i+1$ days." Then the total of days l to r (both included) is

$$a_l + \cdots + a_r = P_{r+1} - P_l.$$

In the picture	In the code
A leg of the trip	One element a_i
The odometer reading	The prefix sum P_i
Odometer at the start: 0	$P_0 = 0$
Distance between two towns	$P_{r+1} - P_l$

With numbers. P&L $= [2, -1, -1, 3, -3, 1]$. The odometer readings are $P = [0, 2, 1, 0, 3, 0, 1]$. Total of days 1 to 3: $P_4 - P_1 = 3 - 2 = 1$. Check: $-1 - 1 + 3 = 1$. Good.

Building P costs $O(n)$ once. After that, **every** range total costs $O(1)$. With a million questions about ranges, that is the difference between 10^6 and 10^{11} steps.

Why $P_0 = 0$ and size $n+1$? It lets a range that starts at day 0 use the same formula, $P_{r+1} - P_0$, with no special case. This "one extra slot at the front" is the most common place to make an off-by-one error.

3.2 The problem

You are given the daily P&L of a strategy as integers $a_0, \ldots, a_{n-1}$ (they can be negative) and a target k. Find the length of the **longest stretch of consecutive days** whose total P&L is exactly k. If there is none, return 0.

Constraints: $1 \le n \le 2 \times 10^5$, $|a_i| \le 10^9$, $|k| \le 10^{15}$.

Examples:

- $a = [2, -1, -1, 3, -3, 1]$, $k = 0$: answer **5** (days 0 to 4: $2 - 1 - 1 + 3 - 3 = 0$).
- $a = [1, -1, 5, -2, 3]$, $k = 3$: answer **4** (days 0 to 3: $1 - 1 + 5 - 2 = 3$).
- $a = [4, 2]$, $k = 1$: answer **0**.

With $k = 0$ this asks for the longest "flat" period: a stretch where the strategy ended where it started.

3.3 Brute force first

Try every start day i. From there, walk the end day j forward, keeping a running total (Lesson 3) so each step costs $O(1)$. Whenever the total equals k, record $j - i + 1$.

That is about $n^2/2$ steps. For $n = 2 \times 10^5$ that is 2×10^{10}: far too slow (a computer does roughly 10^8 to 10^9 simple steps a second).

Can we use two pointers or a window that slides? No. Those need the total to grow when the window grows. Here values can be negative, so a longer stretch can have a smaller total. We need a different idea.

3.4 The insight: turn "a stretch with total k" into "two odometer readings"

A stretch from day l to day r has total k exactly when

$$P_{r+1} - P_l = k \quad \Longleftrightarrow \quad P_l = P_{r+1} - k.$$

In words: "the stretch ending today has total k" means "**some earlier odometer reading equals today's reading minus k.**"

So walk through the days once. At each day, we know today's reading. We ask: **have I seen the reading** $(\text{today} - k)$ **before? Where?** That is a lookup, and Lesson 4 gives us $O(1)$ lookups: a hash map from **reading** to **the day it was seen**.

Longest means earliest. The stretch is longer when the earlier reading is further back. So for each reading we store only the **first** day it appeared, and never overwrite it.

The start reading. Before day 0 the odometer reads 0. We store it at position -1, so a stretch starting at day 0 has length $r - (-1) = r + 1$.

Trace on example 1 ($k = 0$, store reading → day):

Day i	a_i	Reading	Look for reading $-$ 0	Found at	Length	Best	Map after
start		0				0	$\{0: -1\}$
0	2	2	2	no		0	+ $\{2: 0\}$
1	-1	1	1	no		0	+ $\{1: 1\}$
2	-1	0	0	-1	3	3	(0 already there)
3	3	3	3	no		3	+ $\{3: 3\}$
4	-3	0	0	-1	5	5	(keep -1)
5	1	1	1	1	4	5	(keep 1)

Answer **5**. Notice day 4: because we kept the **first** time we saw 0 (day -1), we got length 5, not 2.

Complexity. One pass, one lookup and at most one insert per day: $O(n)$ **time** on average. The map holds at most $n + 1$ readings: $O(n)$ **space**.

Overflow. Readings can reach $2 \times 10^5 \times 10^9 = 2 \times 10^{14}$, far above the `int` limit of about 2.1×10^9. So we use `long long` (up to about 9.2×10^{18}).

3.5 C++ solution

```cpp
// Daily Quant Digest 2026-10-30: Prefix sums.
// Problem: longest stretch of consecutive days whose total P&L is exactly k.
// Compile: g++ -std=c++17 -O2 -Wall 2026-10-30.cpp && ./a.out
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <unordered_map>
#include <vector>

// Warm-up: build the prefix array. prefix[i] = a[0] + ... + a[i-1],
// so prefix[0] = 0 and the total of days l..r (inclusive) is prefix[r+1] - prefix[l].
std::vector<long long> buildPrefix(const std::vector<long long>& a) {
    std::vector<long long> prefix(a.size() + 1, 0);
    for (std::size_t i = 0; i < a.size(); ++i) {
        prefix[i + 1] = prefix[i] + a[i];
    }
    return prefix;
}

long long rangeSum(const std::vector<long long>& prefix, std::size_t l, std::size_t r) {
    return prefix[r + 1] - prefix[l];  // O(1) per query
}

// Brute force: try every start, extend the end, keep a running total. O(n^2).
int longestBrute(const std::vector<long long>& a, long long k) {
    int best = 0;
    const int n = static_cast<int>(a.size());
    for (int i = 0; i < n; ++i) {
        long long sum = 0;
```

```cpp
        for (int j = i; j < n; ++j) {
            sum += a[j];
            if (sum == k) best = std::max(best, j - i + 1);
        }
    }
    return best;
}

// Optimal: one pass with a running total and a hash map that remembers the
// FIRST day each running total was seen. O(n) average time, O(n) space.
int longestWithSumK(const std::vector<long long>& a, long long k) {
    std::unordered_map<long long, int> firstSeen;
    firstSeen.reserve(a.size() * 2 + 1);
    firstSeen[0] = -1;  // the empty start: total 0 "before day 0"

    long long running = 0;
    int best = 0;
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        running += a[i];

        // A stretch ending today with total k needs an earlier running total of running - k.
        auto it = firstSeen.find(running - k);
        if (it != firstSeen.end()) {
            best = std::max(best, i - it->second);
        }

        // Keep only the first time we see this total: the earliest start gives the longest
stretch.
        firstSeen.emplace(running, i);  // emplace does nothing if the key is already there
    }
    return best;
}

int main() {
    // Warm-up check of range sums.
    std::vector<long long> pnl = {2, -1, -1, 3, -3, 1};
    auto prefix = buildPrefix(pnl);
    assert(rangeSum(prefix, 0, 5) == 1);
    assert(rangeSum(prefix, 1, 3) == 1);
    assert(rangeSum(prefix, 3, 4) == 0);

    struct Case { std::vector<long long> a; long long k; int expected; };
    std::vector<Case> cases = {
        {{2, -1, -1, 3, -3, 1}, 0, 5},
        {{1, -1, 5, -2, 3}, 3, 4},
        {{4, 2}, 1, 0},
        {{-2, -1, 2, 1}, 1, 2},
        {{0, 0, 0}, 0, 3},
    };
    for (const auto& c : cases) {
        int got = longestWithSumK(c.a, c.k);
        std::cout << "answer = " << got << " (expected " << c.expected << ")\n";
        assert(got == c.expected);
        assert(longestBrute(c.a, c.k) == c.expected);
    }

    // Randomized cross-check against brute force.
    std::mt19937 rng(20261030);
    std::uniform_int_distribution<int> lenDist(0, 30), valDist(-4, 4), kDist(-6, 6);
    for (int t = 0; t < 20000; ++t) {
        std::vector<long long> a(lenDist(rng));
        for (auto& x : a) x = valDist(rng);
        long long k = kDist(rng);
        assert(longestWithSumK(a, k) == longestBrute(a, k));
    }
```

```
    // Large values: sums need long long.
    std::vector<long long> big(200000, 1000000000LL);
    assert(longestWithSumK(big, 200000LL * 1000000000LL) == 200000);

    std::cout << "All tests passed.\n";
    return 0;
}
```

Output:

```
answer = 5 (expected 5)
answer = 4 (expected 4)
answer = 0 (expected 0)
answer = 2 (expected 2)
answer = 3 (expected 3)
All tests passed.
```

The randomized test compares the fast answer with brute force on 20,000 small random arrays full of negatives and zeros. The last test uses 200,000 days of 10^9 each, whose total (2×10^{14}) would overflow an `int`.

3.6 Walkthrough of the C++

`std::vector<long long> prefix(a.size() + 1, 0);` This constructor takes a **size** and a **starting value**: $n + 1$ slots, all 0. That gives us $P_0 = 0$ for free. Careful: `std::vector<long long> v{5, 0}` with **braces** would make a vector holding the two numbers 5 and 0. Round brackets mean "size and value"; braces mean "these elements".

`for (std::size_t i = 0; i < a.size(); ++i).` `std::size_t` is the unsigned type that `size()` returns, so the comparison causes no `-Wall` warning.

`return prefix;` Returning a whole vector looks expensive, but it is not. C++ **moves** the vector out of the function (or builds it in place), so no element-by-element copy happens.

`static_cast<int>(a.size()).` Converts the unsigned size to an `int` on purpose, so that `i - it->second` can be computed with signed numbers. Why it matters: `it->second` can be -1. If we mixed unsigned and signed here, -1 would turn into a huge positive number and the length would be garbage. `static_cast` is the explicit, searchable way to say "I mean this conversion".

`std::unordered_map<long long, int> firstSeen;` Lesson 4's hash map. Key: an odometer reading. Value: the first day it appeared.

`firstSeen.reserve(a.size() * 2 + 1);` Asks the map to make room up front, so it does not have to grow and **rehash** (move every entry into a bigger table) several times during the loop. Not needed for correctness, a small speed-up.

`firstSeen[0] = -1;` The `[]` operator creates the key if it is missing and gives back a reference to its value, which we set to -1.

`auto it = firstSeen.find(running - k);` `find` returns an **iterator**: a pointer-like object to the matching entry, or `firstSeen.end()` if the key is missing. `auto` saves writing the long type `std::unordered_map<long long, int>::iterator`.

`it->second`. Each map entry is a `std::pair`: `first` is the key, `second` is the value. The arrow `->` reaches through the iterator to the pair, just like with a pointer.

`firstSeen.emplace(running, i);` Inserts the pair **only if the key is not already there**. If it is, nothing changes. That is exactly "keep the first day". Compare with `firstSeen[running] = i;`, which would **overwrite**, keep the **latest** day, and give the **shortest** stretches instead. One character of difference in intent, a completely wrong answer. (`insert({running, i})` would also work.)

Order inside the loop matters. We **look up first, then insert**. If we inserted today's reading first, then with $k = 0$ we would find today itself and get a stretch of length 0.

`struct Case { ... };` **and** `for (const auto& c : cases)`**.** Same pattern as Lesson 4: a small struct for test data, filled with brace initialization, then looped over by const reference to avoid copies.

`std::uniform_int_distribution<int> lenDist(0, 30), valDist(-4, 4), kDist(-6, 6);` One line declaring three distributions. Each call like `valDist(rng)` draws a random whole number in its range (both ends included) using the generator `rng`.

`for (auto& x : a) x = valDist(rng);` Here `auto&` is a **non-const** reference, so writing to `x` changes the element inside the vector. Without the `&`, we would only change a copy and the vector would stay all zeros.

`1000000000LL`. The `LL` suffix makes the literal a `long long`. Then `200000LL * 1000000000LL` is computed in 64 bits. Without the suffixes, `200000 * 1000000000` is computed in `int` and overflows before it is ever stored.

Where else prefix sums appear. Counting stretches with total k (store a **count** per reading instead of the first day), 2D prefix sums for rectangle totals in a grid, and "difference arrays" (the reverse trick) for adding a value to a whole range in $O(1)$. We will meet them later.

---

4. Music: Eagles, "Hotel California" (1976)

Context

"Hotel California" is the title track of the Eagles' album *Hotel California*, released in December 1976. The single came out early in 1977 and went to number 1 on the US Billboard Hot 100. It won the Grammy for Record of the Year. The music was written by guitarist **Don Felder**, and the words by drummer **Don Henley** and guitarist **Glenn Frey**. Henley sings the lead vocal.

The story behind it

Felder recorded a demo on a cassette at a rented beach house in Malibu, with a 12-string acoustic guitar over a simple drum machine pattern. He gave the band a tape of musical ideas, and Henley picked this one out. The band's working title for it was "Mexican Reggae", because of its slow, swaying rhythm.

Henley and Frey then wrote the words. They have described the song as being about excess and the dark side of the American dream, seen through Los Angeles in the 1970s: a dreamlike story about a traveller who checks into a strange hotel. They have also said, many times, that it is not about anything literal, despite decades of rumours.

One well-known detail: the band moved the song into a different key from Felder's demo to suit Henley's voice. And when they recorded the long guitar ending, Henley wanted it to match the demo closely, so Felder had to recreate what he had played on the tape.

Why people love it

It is a story song that feels like a film: a long drive at night, a strange welcome, and a sense of being trapped somewhere beautiful. The words never fully explain themselves, so every listener fills in their own meaning. And then it ends with one of the most famous guitar passages in rock.

What to listen for

- **The opening.** The 12-string guitar figure over a slow beat. It sets a mood before any singing starts, warm but slightly uneasy.
- **The chord pattern.** The verse goes through a long, winding chord pattern that drops in steps and keeps moving away from home before it returns. That feeling of never quite settling matches the story.
- **The harmonies.** The Eagles were famous for vocal harmonies. Listen to how the other voices join Henley in the chorus.
- **The ending.** Don Felder and **Joe Walsh** trade lead guitar lines, then play together in harmony, two guitars playing different notes that fit together, over the same chords as the verse. It goes on for about two minutes and never feels long. Try to follow which guitar is which (they sit in different places in the stereo mix), and notice the moment they come together.
- **The length.** It runs about six and a half minutes, long for a hit single. Notice how the song keeps your attention without a big change of speed or volume.